

04.2 — Daily Ops Runbook

For: The single operator (Venkat + ERPCLaw subagents) keeping Haritha Hospitals ERPNext live. **Goal:** Catch issues early, keep backups healthy, never surprise users with downtime. **Last updated:** 2026-08-29 **Pre-req reading:** 00.2 Architecture Overview, 04.1 Deployment Runbook, 04.3 Disaster Recovery, 04.4 Incident Response

Purpose

Keep the four Haritha Hospitals ERPNext environments (pberpprod, pberpqa, pberpdev, dev-erp) running smoothly day-to-day. Catch silent failures (broken backups, expired SSL, full disks) before they become incidents.

This runbook is **single-operator friendly** — Venkat can do all checks solo, in ~10 min/day. Subagents handle the heavy lifting when available.

Daily checks (10 min, every morning)

Run **after 08:00 IST** (so all 06:00 backup slots have completed). Takes ~10 min.

Step 1 — Heartbeat status (1 min)

```
# SSH into main VPS as vijay
ssh -i /root/.openclaw/ssh_key vijay@144.217.163.228

# All 4 envs' containers running?
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}' \
  | grep -E 'erp-(prod|qa|dev)-(backend|db|scheduler|queue)-1|erpdev-'

# Expected: 16+ rows, all "Up X hours", no "Exited" / "Restarting"
```

What to look for:

- RestartCount > 0 in `docker inspect <container>` → check logs (Lesson #154: container restart is normal after deploy, NOT after incident)
- Up X minutes for prod containers → investigate why they restarted (likely crash, not deploy)
- Missing containers → check `docker ps -a` for stopped ones

Step 2 — Backup completion (3 min)

```
# Latest backup per env (local)
for env in prod qa dev; do
  echo "=== $env ==="
  ssh vijay@144.217.163.228 "ls -lt /home/vijay/backups/$env/ | head -3"
done
```

```
# Expected: most recent dir = TODAY (within last 6 hours)
# 4 slots/day = 00:00, 06:00, 12:00, 18:00 IST
```

If the most recent dir is OLDER than 6 hours → **backup is broken**. Escalate immediately.

```
# Verify the latest backup's SHA256 (Lesson #79: integrity check)
LATEST=$(ssh vijay@144.217.163.228 "ls -t /home/vijay/backups/prod/*.tar.gz | head -1")
ssh vijay@144.217.163.228 "sha256sum $LATEST"

# Compare to offsite
OFFSITE="venkat@135.125.196.35:/home/venkat/pberpprod_backups/${basename $LATEST}"
ssh vijay@144.217.163.228 "ssh -i /home/vijay/scripts/pberpqa_backup_key venkat@135.125.196.35 "sha256sum $OFFSITE"

# Expected: SHA256 matches
```

Step 3 — Offsite sync (1 min)

```
# Most recent offsite backups
for env in pberpprod pberpqa pberpdev; do
    echo "=== $env offsite ==="
    ssh -i /root/.openclaw/ssh_key venkat@135.125.196.35 \
        "ls -lt /home/venkat/${env}_backups/ | head -3"
done
```

Expected: timestamps within last 6 hours (matches local)

If local has fresh backups but offsite is stale → rsync broken. Check cron + key perms (Lesson #93).

Step 4 — Container health deep-dive (2 min)

```
# Container-specific healthchecks
for c in erp-prod-backend-1 erp-prod-db-1 erp-prod-scheduler-1 \
    erp-qa-backend-1 erp-qa-db-1 erp-qa-scheduler-1 \
    erp-dev-backend-1 erp-dev-db-1 erp-dev-scheduler-1; do
    HEALTH=$(docker inspect --format '{{.State.Health.Status}}' $c 2>/dev/null || echo "no-healthcheck")
    RESTARTS=$(docker inspect --format '{{.RestartCount}}' $c 2>/dev/null)
    echo "$c: health=$HEALTH restarts=$RESTARTS"
done
```

Expected: all "healthy", restart=0 (or low from known deploys)

A high RestartCount with no recent deploy → check logs immediately:

```
docker logs --tail 50 erp-prod-backend-1 | grep -iE 'error|traceback|exit'
```

Step 5 — SSL cert expiry (1 min)

```
# All DuckDNS subdomains
for sub in pberpPROD pberpDEV pberpqa; do
    EXPIRY=$(echo | openssl s_client -servername ${sub}.duckdns.org \
        -connect ${sub}.duckdns.org:443 2>/dev/null | \
        openssl x509 -noout -enddate 2>/dev/null | cut -d= -f2)
    echo "${sub}.duckdns.org: expires $EXPIRY"
done
```

Expected: 30+ days remaining

DuckDNS = Let's Encrypt via acme-companion → auto-renews at 30 days. If < 14 days → cert renewal broken.

```
# Force-renew (if needed)
ssh vijay@144.217.163.228 'docker exec nginx-proxy /app/force-renew'
```

Step 6 — DB password verification (quarterly only, see below)

Skip on daily. Schedule for monthly.

Daily summary template

```
**Date:** 2026-08-29
**Time:** 08:30 IST
**Operator:** Venkat
```

```
| Check | Result |
| --- | --- |
| All containers up | ☐ |
| Backup slot 06:00 IST (prod) | ☐ 1.4MB tar.gz |
| Offsite sync (prod) | ☐ venkat VPS, SHA match |
| Container health (all 9) | ☐ all healthy, restart=0 |
| SSL cert expiry | ☐ 67 days remaining |
| disk space | ☐ main 83%, venkat 60% |
| Issues today | none |
```

Append to memory/YYYY-MM-DD.md.

Weekly checks (15 min, Monday morning)

DB password verification (Lesson #154)

DB passwords can drift. Verify all 4 envs against container env vars:

```
for env in prod qa dev; do
  DB="erp-${env}-db-1"
  echo "=== $env ==="
  echo "MEMORY: $(grep -A1 'pberp${env} DB password' /root/.openclaw/workspace/MEMORY.m
  echo "Container: $(docker exec $DB printenv MYSQL_ROOT_PASSWORD)"
done
```

```
# dev-erp uses different prefix
echo "=== dev-erp ==="
echo "Container: $(docker exec erpdev-mariadb-1 printenv MYSQL_ROOT_PASSWORD)"
```

If mismatch → update MEMORY.md IMMEDIATELY (don't trust old literals). Lesson #154: hard-coded passwords have drifted twice (2026-08-29 prod incident).

Disk space check

```
# Main VPS
ssh vijay@144.217.163.228 'df -h / | tail -1'
# venkat VPS (offsite)
ssh -i /root/.openclaw/ssh_key venkat@135.125.196.35 'df -h / | tail -1'
```

```
# Expected: < 85% on both
```

If > 90% → check /var/lib/docker (Docker volumes accumulate):

```
ssh vijay@144.217.163.228 'docker system df'
# Reclaim if needed (Lesson #91):
ssh vijay@144.217.163.228 'docker image prune -a'
ssh vijay@144.217.163.228 'docker system prune'
```

Frappe error logs (tabError Log)

```
# Errors in last 24h per env
for env in prod qa dev; do
  echo "=== $env ==="
  docker exec erp-${env}-db-1 bash -c "
    mariadb -u root -p\"$(printenv MYSQL_ROOT_PASSWORD)\" \\\
    -D _${env}_db \\\
    -e \"
      SELECT method, COUNT(*) as cnt, MAX(timestamp) as latest
      FROM `tabError Log`
      WHERE timestamp > DATE_SUB(NOW(), INTERVAL 24 HOUR)
      GROUP BY method
      ORDER BY cnt DESC
      LIMIT 10;
    \"
  done
```

Expected: low counts (< 10/day), no new method names

A spike in tabError Log → investigate before users notice.

Cron line integrity (Lesson #78, #92)

```
ssh vijay@144.217.163.228 'crontab -l | grep -c "backup.sh"'
# Expected: 4 (one per env + dev-erp)
```

Compare to expected count from MEMORY.md

If count dropped → cron got clobbered. Restore from git-tracked backup or rebuild via heredoc pattern (Lesson #77).

Monthly checks (30 min, first of month)

Review LEARNINGS.md for new lessons

```
# Lessons added in last 30 days
grep "^### #1[5-9][0-9]\|^### Lesson #1[5-9][0-9]\|^### Lesson #2[0-9][0-9]" /root/.openclaw

# Read each, decide:
#   - incorporate into runbook (this doc or 04.1/04.3/04.4)
#   - add to MEMORY.md standing rules
#   - keep as reference only
```

Update MEMORY.md

If any of these changed:

- ☐ DB passwords (quarterly — see above)
- ☐ New envs added/removed
- ☐ New apps installed (e.g. haritha_hospital added 2026-08-29)
- ☐ Container names changed
- ☐ Backup destinations changed
- ☐ New cron entries

DR drill (Lesson #48: quarterly)

Actually restore from backup to a scratch site. Verify the bundle is restorable.

Procedure: see 04.3 Disaster Recovery §“Test cadence”.

Test backup restore (subset)

Don't full-restore every month — too disruptive. Instead, restore a SINGLE table to verify the SQL is parseable:

```
LATEST=$(ssh vijay@144.217.163.228 "ls -t /home/vijay/backups/prod/*.tar.gz | head -1")
TMPDIR=$(mktemp -d)
ssh vijay@144.217.163.228 "tar -xzf $LATEST -C $TMPDIR --wildcards '*-database.sql.gz'"
DB_FILE=$(ls $TMPDIR/*-database.sql.gz)
gunzip -t $DB_FILE && echo "OK: gzip layer" || echo "ERROR: gzip corrupt"
head -100 $DB_FILE | grep -c "CREATE TABLE" # expect > 0
rm -rf $TMPDIR
```

Annual: rotate SSH keys

If using the same /root/.openclaw/ssh_key for > 1 year, consider rotating. Current keypair is still in use; rotation pending user approval.

Common commands

Container operations

Restart a single container (preserves volumes, Lesson #154)

```
docker restart erp-{env}-backend-1
docker restart erp-{env}-db-1
docker restart erp-{env}-scheduler-1
```

Inspect a container

```
docker inspect erp-prod-backend-1 | jq '[] | {Name, State, RestartCount}'
```

View logs

```
docker logs --tail 100 erp-prod-backend-1
docker logs --tail 100 -f erp-prod-backend-1 # follow mode
docker logs --since "2h ago" erp-prod-backend-1
```

Resource usage

```
docker stats --no-stream | grep erp-
```

Execute inside container

```
docker exec erp-prod-backend-1 bash
docker exec -u root erp-prod-backend-1 bash    # if root needed
```

Bench operations

Restart bench (only re-runs Frappe hooks, NOT gunicorn – Lesson #46, #153)

```
docker exec erp-prod-backend-1 bash -c '
  cd /home/frappe/frappe-bench &&
  bench restart'
```

Clear cache (after schema/migration changes)

```
docker exec erp-prod-backend-1 bash -c '
  cd /home/frappe/frappe-bench &&
  bench --site pberpprod.duckdns.org clear-cache'
```

Migrate (idempotent – safe to re-run)

```
docker exec erp-prod-backend-1 bash -c '
  cd /home/frappe/frappe-bench &&
  bench --site pberpprod.duckdns.org migrate'
```

Console (Python REPL with Frappe context)

```
docker exec erp-prod-backend-1 bash -c '
  cd /home/frappe/frappe-bench &&
  bench --site pberpprod.duckdns.org console'
```

Note: Lesson #95 – no '-c' flag in v16; use stdin redirect

Backup (Lesson #79 pattern with timeout)

```
docker exec erp-prod-backend-1 bash -c '
  cd /home/frappe/frappe-bench &&
  timeout 900 bench --site pberpprod.duckdns.org backup --with-files'
```

Export fixtures (Rule #9, Lesson #151)

```
docker exec erp-prod-backend-1 bash -c '
  cd /home/frappe/frappe-bench &&
  bench --site pberpprod.duckdns.org export-fixtures --app haritha_hospital'
```

MySQL queries

Always use env var pattern (Lesson #154: never hardcode)

```
DB_PASS=$(docker exec erp-prod-db-1 printenv MYSQL_ROOT_PASSWORD)
```

```
docker exec erp-prod-db-1 mariadb -u root -p"$DB_PASS" -D _b80f05e76a0dcaad \
  -e "SELECT name, module FROM \`tabCustom Field\` WHERE module='haritha_hospital' LIMIT 1"
```

Quick health: row counts per table

```
docker exec erp-prod-db-1 mariadb -u root -p"$DB_PASS" -D _b80f05e76a0dcaad \
  -e "SELECT
    (SELECT COUNT(*) FROM \`tabEmployee\`) AS employees,
    (SELECT COUNT(*) FROM \`tabAttendance\`) AS attendance,
    (SELECT COUNT(*) FROM \`tabShift Assignment\`) AS shift_assignments,
    (SELECT COUNT(*) FROM \`tabError Log\` WHERE timestamp > DATE_SUB(NOW(), INTERVAL
```

SSH tunnel for direct DB access

```
# Forward local port 3306 to remote MariaDB through VPS
ssh -i /root/.openclaw/ssh_key -L 3306:erp-prod-db-1:3306 vijay@144.217.163.228 -N

# Then locally:
mysql -h 127.0.0.1 -P 3306 -u root -p # password from container env
```

Disk / Docker cleanup

```
# List reclaimable space
docker system df

# Reclaim (Lesson #91: schedule weekly)
docker image prune -a          # remove unused images
docker container prune        # remove stopped containers
docker volume prune           # CAREFUL – only anonymous volumes
docker network prune          # unused networks

# Never run `docker system prune -a --volumes` – would delete named volumes
```

Git operations (for haritha_hospital repo)

```
# Workspace
WS=/home/vijay/frappeclaw/frappeclaw-data/workspace/projects/haritha-hospitals
cd $WS

# Pull latest
git checkout main && git pull origin main

# Status
git status

# Create feature branch
git checkout -b feature/my-change

# Push (always as venkat-narasimha – Rule #11)
git config user.name venkat-narasimha
git config user.email srivenkatnarasimha@gmail.com
git commit -m "...
git push origin feature/my-change
```

Alerts & escalation

Auto-monitored (subagents)

Signal	Threshold	Action
Backup slot missed	no new bundle by 06:30 IST	Subagent: investigate cron + log
tabError Log spike	> 50 errors/hour	Subagent: triage, alert if P1

Signal	Threshold	Action
Container restart	RestartCount increase	Subagent: check logs, notify if prod
Disk > 90%	df shows > 90%	Subagent: prune, alert
SSL expiry < 14 days	cert check	Subagent: force-renew

Operator alerts (you ping Venkat)

Severity	Trigger	Response time
SEV-1	Total outage (HTTP 5xx on prod login)	wake Venkat immediately
SEV-2	Degraded (1 feature broken, workaround exists)	notify within 30 min
SEV-3	Minor (cosmetic, no user impact)	notify in daily summary
SEV-4	Cosmetic (typo, alignment)	back of queue

Auto-fix (no ping needed)

- Disk cleanup (Lesson #91, weekly cron)
- Backup verification (subagent at slot time)
- Cert auto-renew (Let's Encrypt)
- Container restart from known-good deploy

Escalation matrix

Dev env issue → try yourself (no ping)
 QA env issue → try yourself; if user-blocking, ping Venkat
 Prod issue → ALWAYS ping Venkat (Telegram)
 Data loss risk → STOP all changes, ping Venkat IMMEDIATELY

On-call rotation

Single-operator model — no formal rotation. But backup contacts:

Role	Person	Contact	Hours
Primary	Venkat	Telegram @Miles_Morales_12	24/7 (single point of failure)
Backup	ERPCLaw subagents	via OpenClaw sessions_spawn	automated
Vendor support	Frappe (community forum)	https://discuss.frappe.io/asyn	

If Venkat unreachable for > 1 hour during a SEV-1:

1. Check Venkat Telegram for “out of office” / auto-reply
2. If truly unreachable, attempt auto-recovery only (restart container, no schema changes)

3. Document incident + state; escalate to Venkat when available

This is a **single-operator environment**, not a production team. Backup contacts are advisory, not 24/7 SLA.

Automation wishlist (deferred)

- ☐ Cron-driven daily summary email to Venkat
- ☐ Slack/Telegram webhook on tabError Log spike
- ☐ Auto-restart on unhealthy container (compose healthcheck + restart: unless-stopped)
- ☐ Backup verification cron (parse *.sha256 against offsite)
- ☐ Disk space alert (cron + Telegram)

These are “nice to have” — current manual cadence is sustainable.

References

- **Architecture:** 00.2 Architecture Overview
- **Backup scripts (Lesson #79 hardened):** /home/vijay/scripts/{pberp,dev,qa,prod}_backup.sh
- **Lessons cited:** #46 (install-app restart), #48 (8-phase test), #77 (atomic crontab), #78 (concurrent cron), #79 (timeout backup), #91 (image prune), #92 (no write append), #93 (ssh key chmod), #95 (no bench console -c), #151-#157 (custom app + fixtures)
- **MEMORY.md:** tech stack, env access, backup cron schedule, container names
- **Disaster recovery:** 04.3 Disaster Recovery
- **Incident playbook:** 04.4 Incident Response

Operator discipline > automation. If unsure, check it manually.